Titanfall Movement System (TMS) – Advanced FPS Movement for Unreal Engine

Thank you for showing interest in the Titanfall Movement System for Unreal Engine!

TMS is a first-person movement system designed for fast-paced shooter games. Inspired by the fluid mechanics of *Titanfall* and *Apex Legends*, TMS delivers a physics-based, high-mobility experience with mechanics that feel both smooth and responsive. After months of research, development, and discussions with top Titanfall speedrunners, TMS was born: a system that faithfully replicates the ***feel of Titanfall & Source Engine movement*** within Unreal Engine.

This documentation will walk you through the **basics of the system**, providing detailed explanations of each **movement mechanic** and **function**. It also serves as a **reference guide** for adjusting the **tweakable variables** found in both the **PlayerCharacter** and the **GrapplingHook Actor**.

English isn't my native language, so if there's any misspellings let me know!

Contents

The Player Character

The BP_TMSPlayerCharacter serves as the base of the movement system. This character inherits from the Unreal Engine Character and handles both the inputs and logic of TMS.

In order to use the TMS Character in your existing project, it is best to change the inheritance of your custom player character to the BP_TMSPlayerCharacter. This will allow your custom character to call functionality of the TMSPlayerCharacter, while keeping its own systems isolated from the TMS. Input Mappings for the mechanics of TMS are implemented via the Enhanced Input Mapping Context found in the "Inputs" Folder, and can be overwritten in child characters to disable mechanics and add your own functionality to them. You could also only copy over specific functions, variables and macros to add only parts of the movement code, which takes a little longer and requires a basic understanding of the structure of TMS. *Reading through the Documentation can help with that :)!*

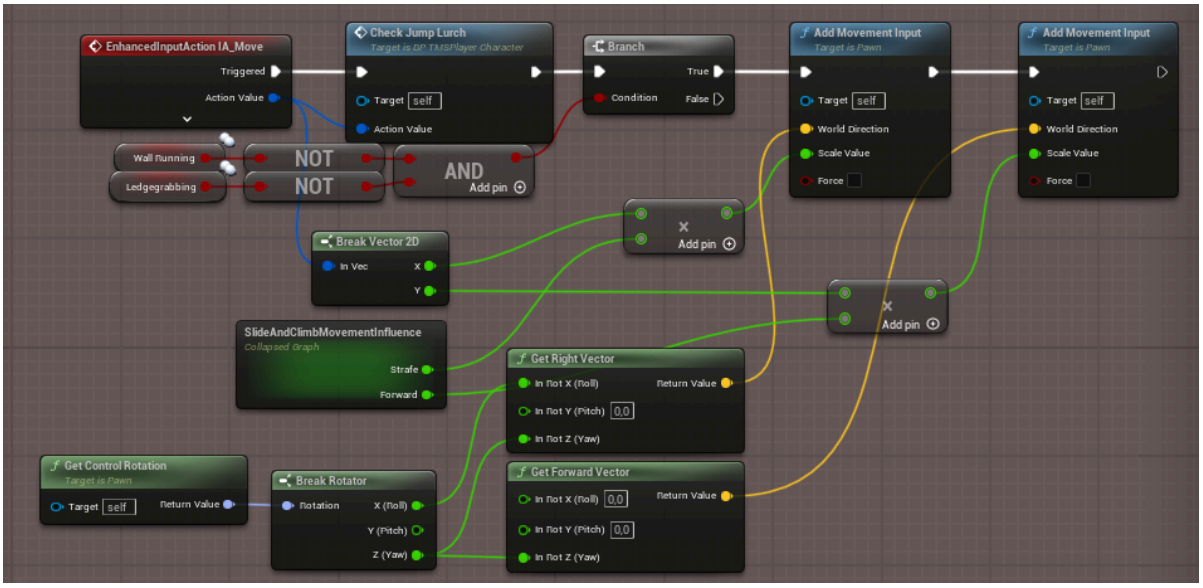
In the following we will go over each movement mechanic: how it works, how it's implemented, and how to tweak it to your liking. We will first look at the

implementation, and then give a concise chart of any adjustable variables at the end of each mechanic.

Move and Look Inputs

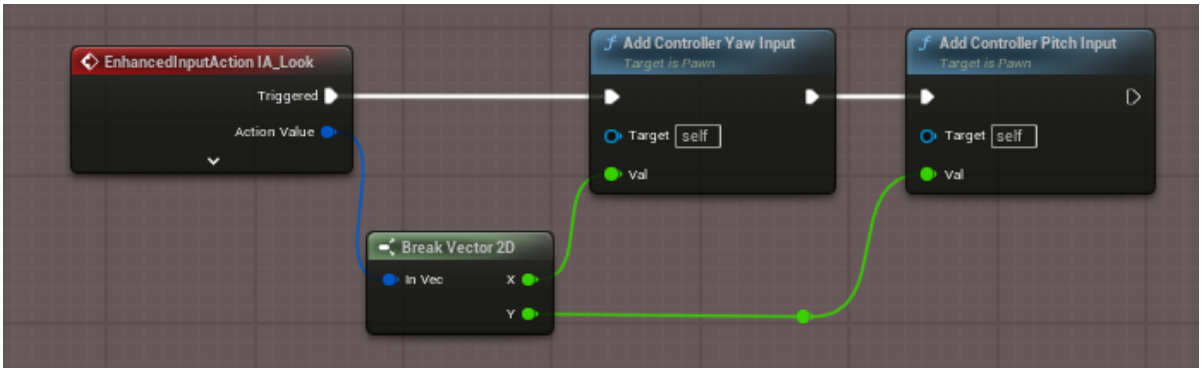
The basis for any movement of a character are the Movement and Looking Controls. These are handled via the IA_Move and IA_Look Input Actions.

IA_Move:



The IA_Move Implementation is very straightforward. It checks for any triggers of the *Jump Lurch* mechanic, which will be explained in detail later. It then runs a few checks to see if we are able to move (not the case while *Wallrunning* or *Ledgegrabbing* since their movement is handled elsewhere). The only really notable addition is the Collapsed “Slide and Climb Movement Influence” Graph. This graph handles the movement multipliers while *Sliding* and *Climbing.* While performing either of these actions, we want to limit the sideways strafing movement of our character quite heavily, since the slide should mainly force the player forward and climbing should mainly be happening vertically and not horizontally.

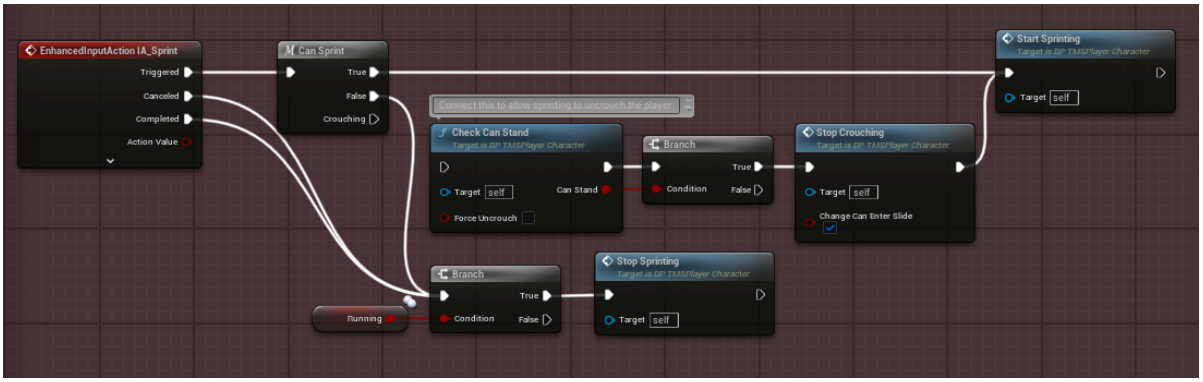
IA_Look:



The IA_Look Implementation is the basic implementation for turning the camera, and will not be explained in detail.

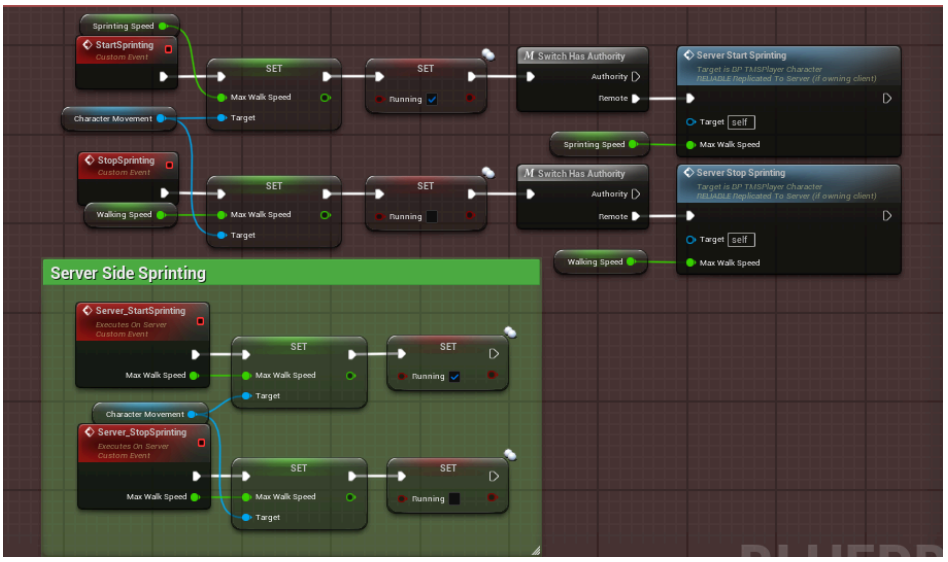
Sprinting Input

IA_Sprint:



The IA_Sprint Implementation is also quite simple. We first check to see if sprinting is possible. This is influenced by our character's "falling" state, if we are crouching and if we are holding W to move forward. If we can sprint, call the "Start Sprinting" Event, and call "Stop Sprinting" if we are unable to sprint. If you want sprinting to be able to cancel crouching, hook up the "Crouching" Output Pin of the CanSprint Macro to the "Check Can Stand" Function.

Start and Stop Sprinting, and Server Side Replication:



The Start and Stop Sprinting Events are responsible for changing our character's movement speed, the "Running" Boolean, and are also replicated across all clients by setting their values on the server side as well. Change the "Sprinting Speed" Variable to control the velocity the character has while sprinting.

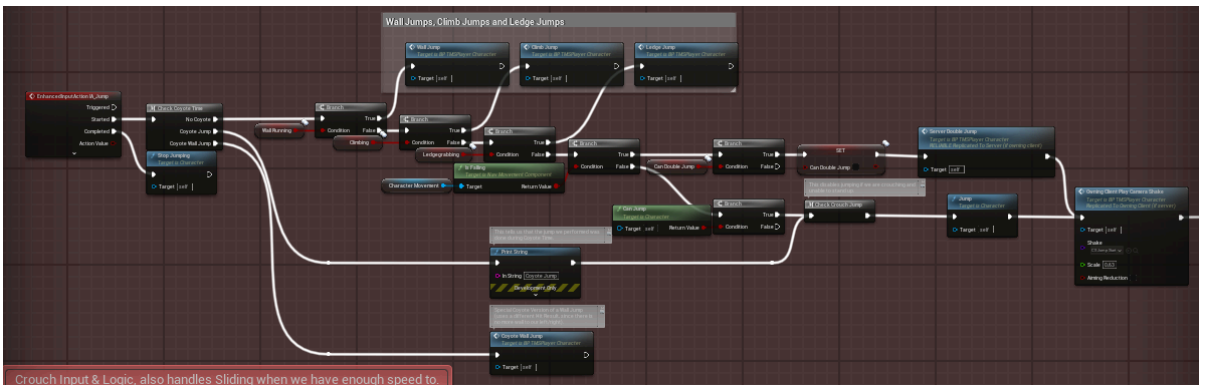
This concludes the Sprinting Implementation. The variables used are the following:

Variable Name	Variable Type	Description	Default Value
SprintingSpeed	float	Dictates the movement speed the character will have when sprinting.	1150 Units
bRunning	boolean	Keeps a reference of our character's running state.	false

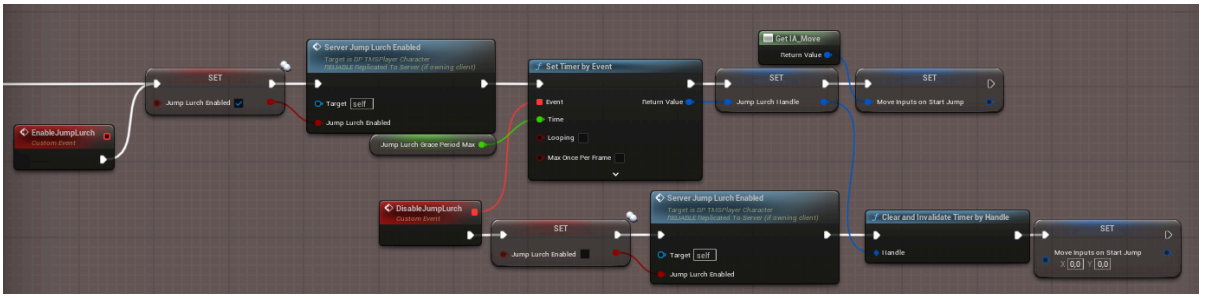
Jumping Input

The jumping input is the first somewhat complex mechanic. It handles *Coyote Time* jumping, wall- climb- and ledge jumps, double jumps and activates *Jump Lurch.*

IA_Jump:



This portion of code is preferably left unchanged. It handles differentiating between *Coyote Time* and grounded jumps, and also triggers a double jump if it is available. The main thing that can be changed to your liking is the “Can Jump” Method, which is slightly overwritten in order to enable *Coyote Time* to work.



The method continues by enabling Jump Lurch, a mechanic that allows for sudden directional changes for a short period after jumping. This mechanic is prevalent in Titanfall 2, and a very important cornerstone of the movement. It can be beneficial by increasing maneuverability while in the air, but can also hurt the speed a player has, since it decreases the velocity by “JumpLurchSpeedLoss(in %)”. Jump Lurch is one of those things that is very noticeable when starting out, and can even feel a little unfamiliar at times, but mastering it is a crucial part in Titanfall Speedrunning, and increases the skill ceiling of a game, while also helping newer players by allowing them to correct directional mistakes after jumping.

The actual application of Jump Lurch happens whenever we input any movement commands. I have already hinted at the “Check Jump Lurch” function in the IA_Move section, so feel free to check it out in-engine. In a nutshell, this function checks if we are pressing a different movement key then when we started jumping (WASD), if we are, we are going to apply a launch velocity in that direction, and also decrease the player’s speed by the aforementioned “JumpLurchSpeedLoss(in %)” Variable.

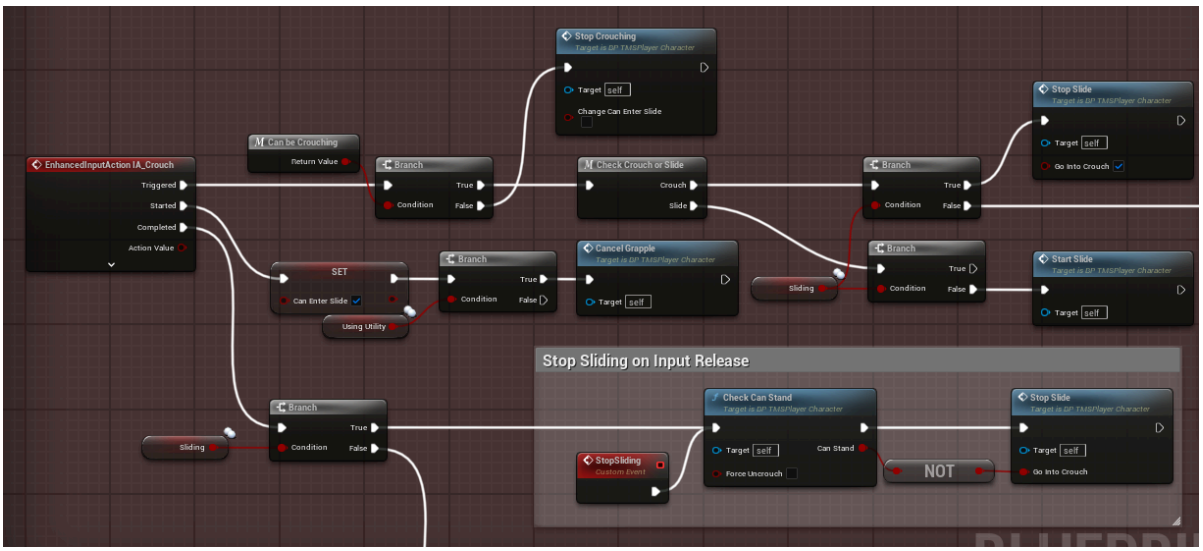
This concludes the Jumping Implementation. The variables used are the following:

Variable Name	Variable Type	Description	Default Value
bCanDoubleJump	boolean	Keeps a reference of if our character is able to double jump. This gets reset upon landing on the ground or wallrunning.	true
bJumpLurchEnabled	boolean	Keeps a reference of if we should try to apply jump lurch when inputting movement. This gets reset after a short duration.	false
JumpLurchGracePeriodMin	float	Is relevant for the strength of our jump lurch. The closer we are to this variable, the stronger the lurch will be. If we are faster than this, the lurch has its highest strength.	0,2 secs
JumpLurchGracePeriodMax	float	Is relevant for the strength of our jump lurch. The closer we are to this variable, the weaker the lurch will be. If this time passes, lurch will be disabled entirely. When trying to preserve momentum, it is important to wait this long before inputting movement as to not trigger jump lurch.	0,5 secs
JumpLurchStrength	float	Is relevant for how strong our lurch is in general. Higher values mean a stronger lurch. This value is quite arbitrary, but defaults to 5 – as is in Titanfall 2.	5 Units
JumpLurchMax	float	Limits how strong any jump lurch can be. It essentially clamps the launch magnitude to its value.	450 Units
JumpLurchVelocity	float	Plays the main role in how strong any jump lurch can be. This velocity is taken as base value, and then scaled on how fast we pressed any movement key after jumping, being multiplied by JumpLurchStrength in the process, while also being clamped by JumpLurchMax.	100 Units
JumpLurchSpeedLoss (in %)	float	Defines how much speed we lose when lurching. Plays a key role in making lurch have a tradeoff between maneuverability and speed.	12,5%
JumpLurchHandle	Timer Handle	Don't modify. Is the handle for the lurch disable timer.	-
MoveInputsOnStartJump	Vector2D	Don't modify. Stores a reference of the movement inputs the character had when jumping. This plays a key role in allowing jump lurch in certain directions.	-

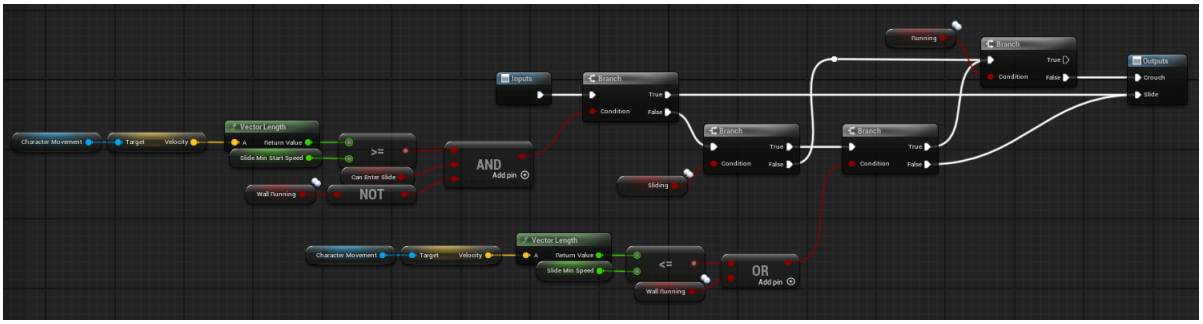
Crouching Input

The crouching input handles, well… crouching. But also differentiates between crouching and sliding. Since both mechanics utilize this input, this is quite a large and complex mechanic. But I will try my best to explain each of the underlying systems.

IA_Crouch:



This portion of code handles the differentiation between sliding and crouching, and enables/disables the other based on it. The “Can be Crouching” Macro only really checks if we are currently “falling”, or using the grappling hook (“Using Utility”). If we aren’t we can continue either sliding or crouching. The “Check Crouch or Slide” Macro does exactly what the name suggests: it checks if we should be sliding or crouching.



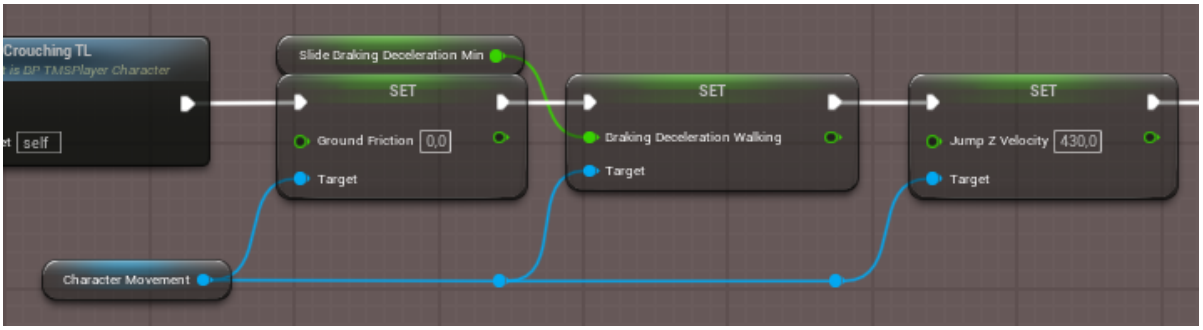
If we can slide depends on multiple factors, mainly that we aren’t wallrunning, our speed is greater than “Slide Min Start Speed” and we can enter a slide in the first place. This Macro also cancels sliding when we are slower than “Slide Min Speed”, transitioning the end of a slide into a crouch seamlessly.

I will not go into complete detail over everything the IA_Crouch Input handles, but in a nutshell it sets the movement speed of our player to the “Crouch Speed”, sets the “bCrouching” boolean to true and also smoothly decreases our capsule half height to “Crouching Capsule Half Height”.

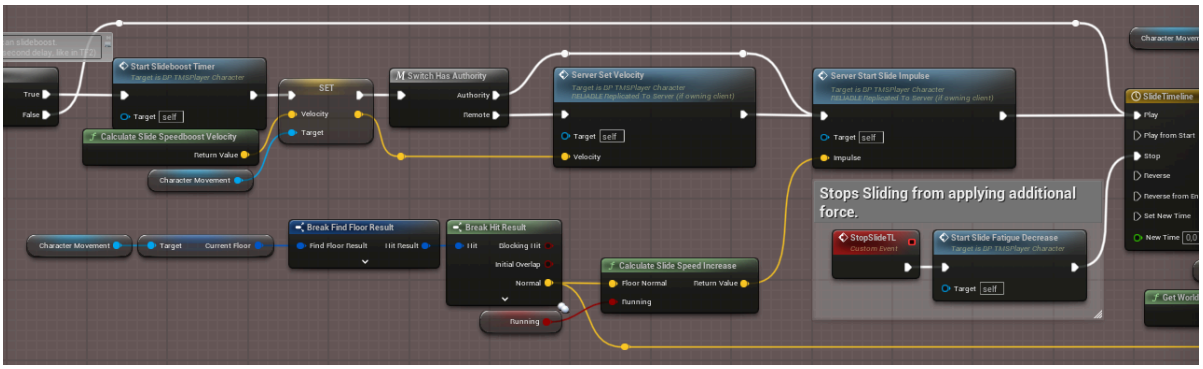
An important part in being able to uncrouch is the “Check Can Stand” function. This function capsule traces for any obstructing objects above us. If something does obstruct our capsule, and we aren’t fully able to stand this function disallows us to

stand up, and continuously gets checked while the crouching input isn't pressed, uncrouching us the moment we have enough space to stand up.

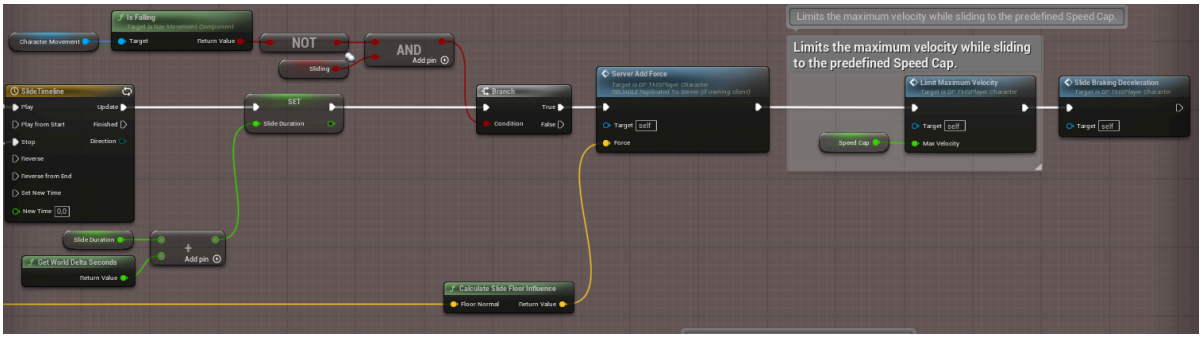
The sliding portion of code does a lot of things at once, but the main parts are the following:



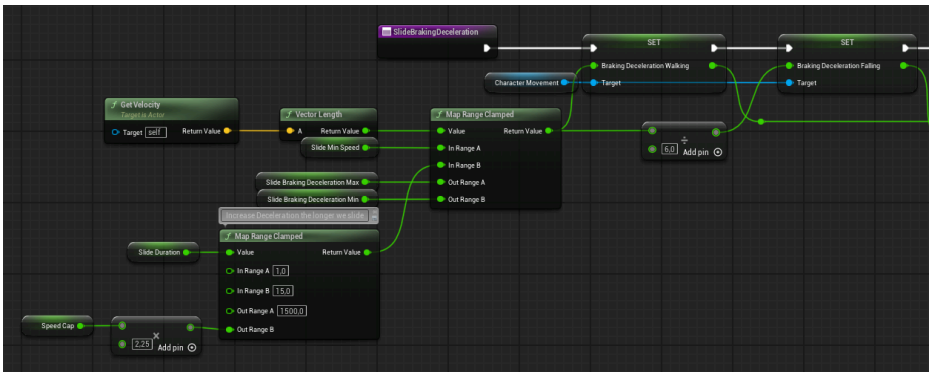
It decreases the ground friction to 0, and also decreases the braking deceleration and jump height since slidehops shouldn't be as high as regular jumps.



It then gives us a small impulse when starting the slide, but only if we haven't slid for the past 2 seconds. (is a restriction in Titanfall 2, controlled by the "bSlideboostTimer" boolean and "Slideboost Cooldown Duration" float values)



It then smoothly adds force to our character, taking the surface angle and time we slid into consideration. The force gradually slows down as more time passes. This is handled by the "Slide Braking Deceleration" function at the end here. That function looks something like this:



It increases the braking deceleration the longer we slide. This has the effect of slowing down the player the longer we are sliding. The air deceleration is also decreased, so that slidehopping – while preserving velocity for longer – cannot go on indefinitely.

This concludes the Crouching and Sliding Implementation. The variables used are the following:

Variable Name	Variable Type	Description	Default Value
bCrouching	boolean	Keeps a reference of our character's crouching state.	false
CrouchAlpha	float	Don't modify. Is used for smoothly shrinking down the players capsule half height and view height.	-
CrouchingSpeed	float	Dictates the movement speed the character will have when crouching. Since we aren't using the CMCs included crouching system, we need to set this speed here.	400 Units
UncrouchHandle	Timer Handle	Don't modify. Is the handle for the "Check Can Stand" timer. Is responsible for uncrouching the player as soon as he can stand and isn't holding the crouch input.	-
bSliding	boolean	Keeps a reference of our character's sliding state.	false
SlideMinStartSpeed	float	This is the minimum speed the player needs to have to be able to slide.	850 Units
SlideMinSpeed	float	This is the speed at which the slide cancels itself. Is set to a lower value to incentivise players to not slide until the end.	225 Units
SlideMinSpeedBurst	float	This is the minimum speed boost the player gains when entering a slide. The speed boost is determined between this and the following variable, and the speed the player was when entering the slide.	100 Units

Variable Name	Variable Type	Description	Default Value
SlideMaxSpeedBurst	float	This is the maximum speed boost the player gains when entering a slide.	400 Units
SlideBrakingDecelerationMin	float	This is the minimum braking deceleration the player experiences while sliding. Responsible for slowing down the player. This gradually increases to the following variable the longer the player is sliding, to avoid them sliding all over the place. Also affects the falling braking deceleration, though less.	375 Units
SlideBrakingDecelerationMax	float	This is the maximum braking deceleration the player experiences while sliding. Responsible for slowing down the player. Also affects the falling braking deceleration, though less.	750 Units
bSlideboostTimer	boolean	When this is true, the player will not receive a boost to speed when starting a slide. The duration this persists between slides is set by the following variable.	false
SlideboostCooldownDuration	float	This influences how long the player is locked out of receiving a slideboost between slides. Exists in Titanfall 2, and exists here.	2,0 secs
bCanEnterSlide	boolean	Keeps a reference on whether or not our player can enter a slide.	true
SlideFatigueCounter	int	This keeps track of slide fatigue. Slide fatigue is important for slidehopping. When landing during a slide, our player gains a small burst of movement speed. This counter is used for diminishing returns on slidehopping, making spamming of the jump button while sliding less incentivising.	0
SlideFatigueHandle	Timer Handle	Don’t modify. Is the handle which gradually resets the Slide Fatigue Counter after we stopped sliding.	-
SlideFloorInfluenceForce	float	Influence the floor has on the slide direction. Higher values mean the floor drags the player more, making it impossible to slide uphill. This is advised to remain unchanged, as force values are pretty arbitrary.	850.000 Units
SlideDuration	float	Don’t modify. Keeps track of how long the player has been sliding without interruption. Is used for the sliding deceleration.	-

Coyote Time

Coyote Time is a very simple, but effective mechanic that grants players a short period in which they can still input jump actions even after falling off a ledge or a wall. This makes movement feel a lot more responsive and less punishing. Coyote Time works for both regular jumps on the ground, as well as wall jumps. The implementation is really quite simple, so I won’t go over the details. It is implemented under the “On Movement Mode Changed” event.

The variables used to control coyote time are the following:

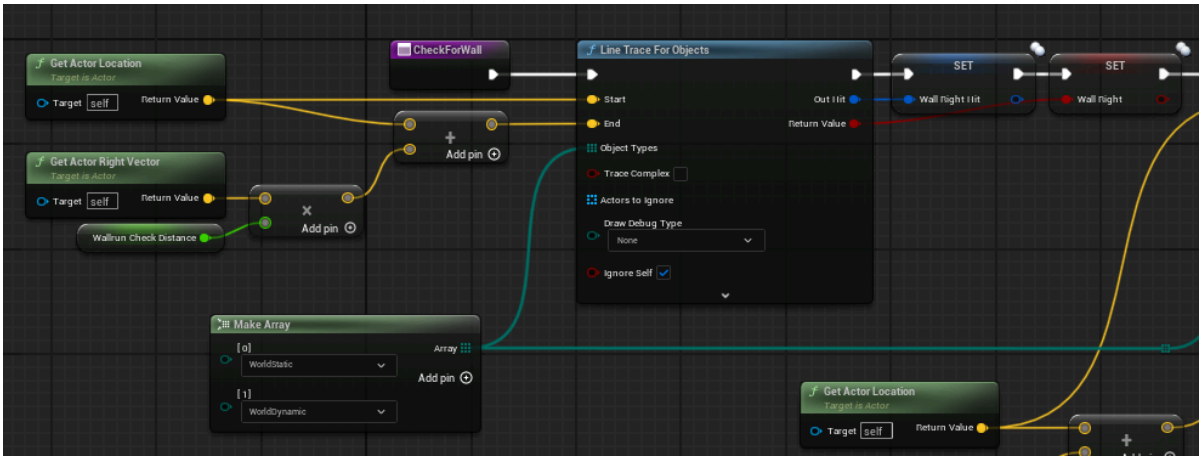
Variable Name	Variable Type	Description	Default Value
CoyoteTime	float	This is the maximum duration coyote time can last. Coyote time scales with movement speed, so the faster you were going, the longer the grace window is.	0,165 secs
CoyoteTimeHandle	Timer Handle	Don't modify. Is the handle which keeps a reference of the current coyote time. If this handle is active, it means that coyote time is active.	-

Wallrunning

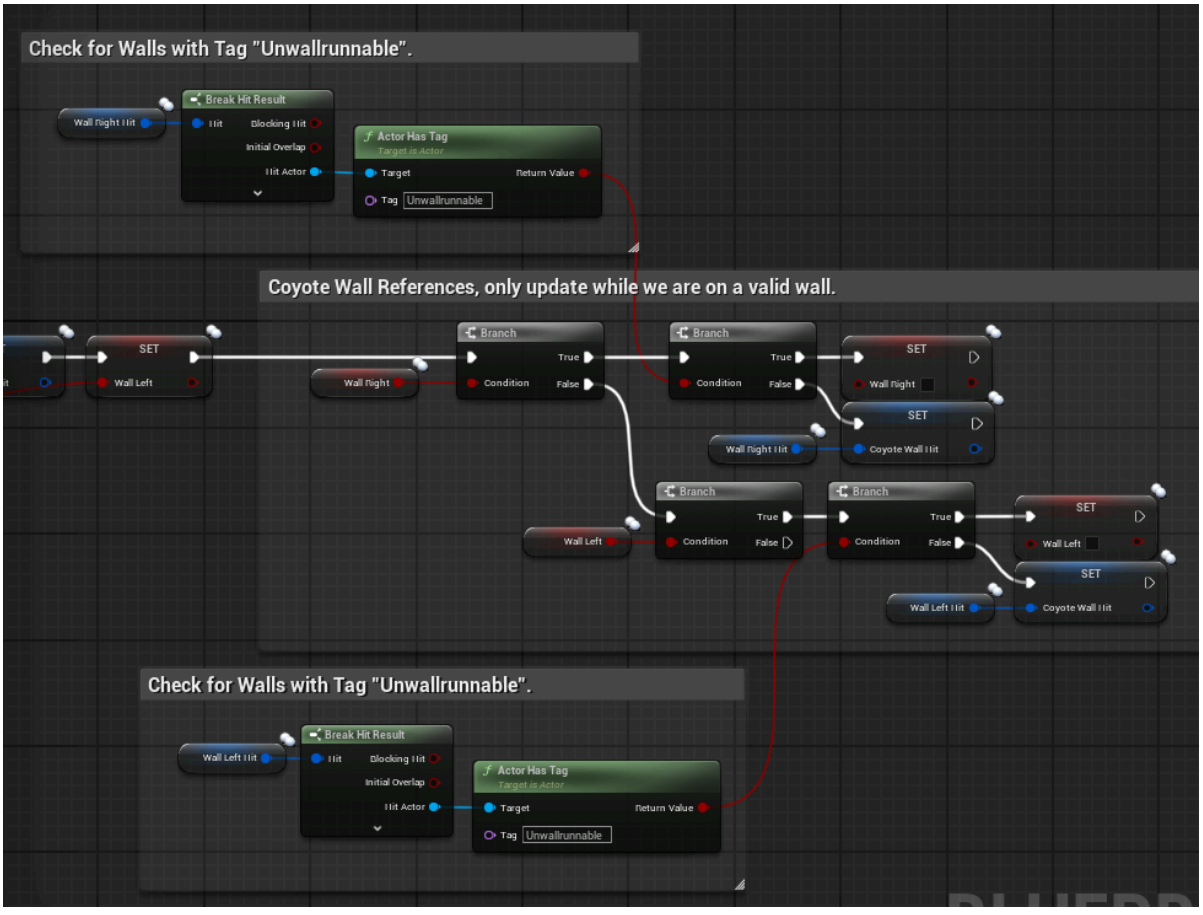
The cornerstone of any advanced movement system is arguably wallrunning. If programmed correctly, it can be a great addition to any game, incentivising movement and offering a smooth feel while on the wall. If implemented sluggishly, it can feel bad and make games even more restricting than not having it in the first place. TMS uses a physics-based system to move the player alongside the wall, fully compatible with curved collisions and movable objects. While the system is not too complex on the coding side, understanding what is going on behind the scenes and the math involved can be somewhat difficult. I will attempt to explain what each calculation does, and how the adjustable variables impact specifics of the wallrun.

The actual logic of the wallrunning happens during Event Tick. Wallrunning is split up into three functions. The first one being:

Check for Wall:

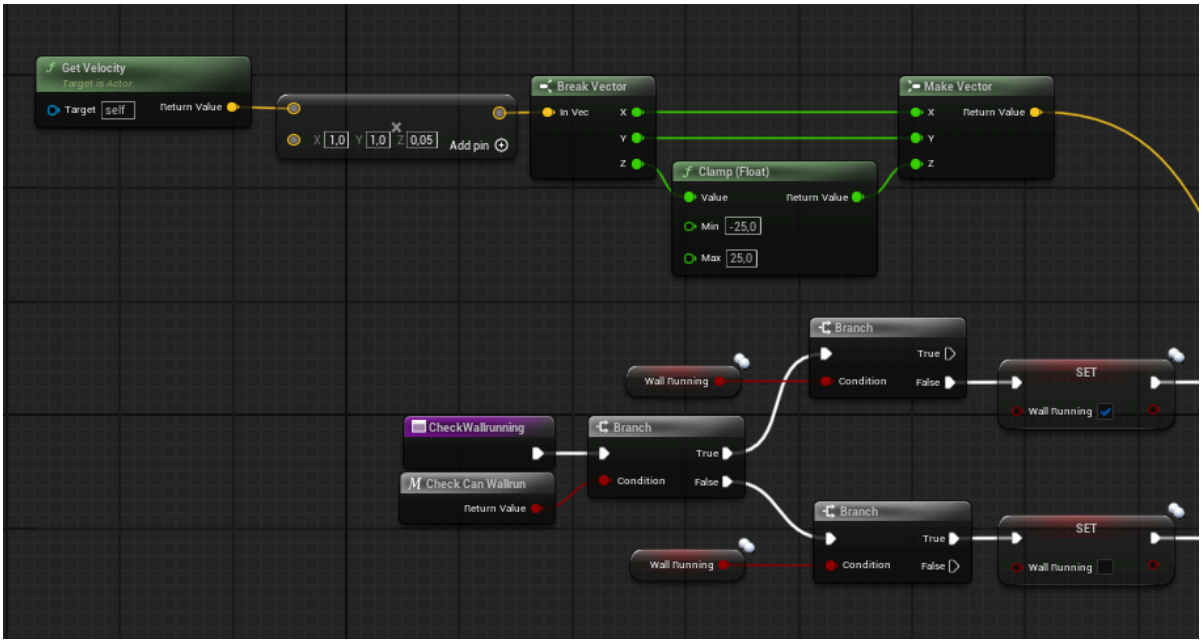


This function checks the left and right side of the player for any walls that could be a potential wallrun target. These walls need to be of the Object Types fed into the Line Traces via the Make Array (Default: WorldStatic and WorldDynamic).

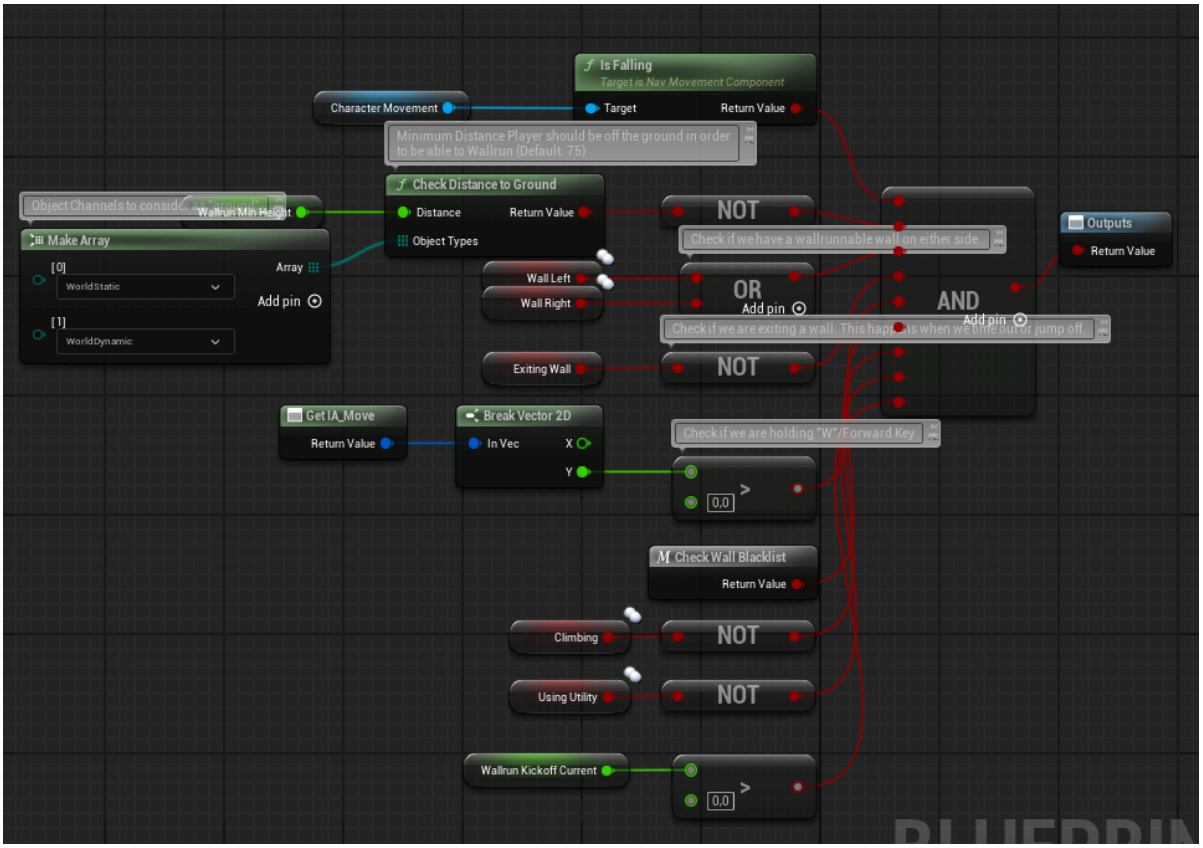


The function then checks if the walls we have to our sides have the tag “Unwallrunnable”. This tag acts as the blacklist for walls. Adding it to any object or component in the scene completely disallows wallrunning. The function also stores a “Coyote Wall Hit” Result, that is used for wall jumps when coyote time is active, since we do not have a wall to either side at that point.

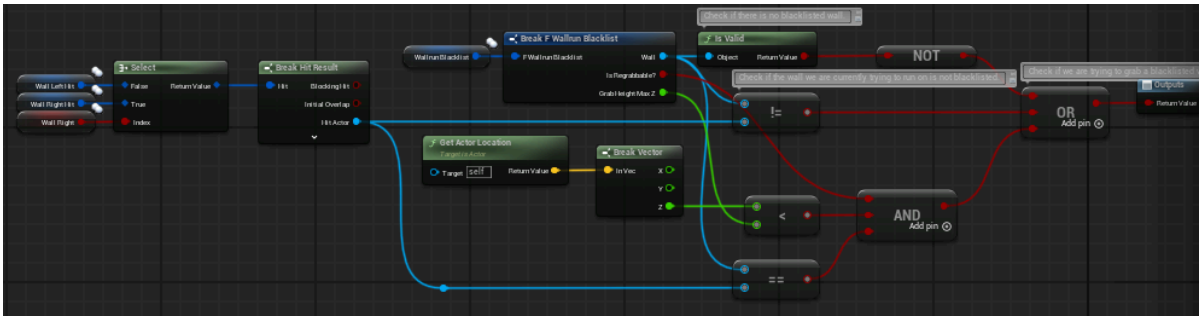
Check Wallrunning:



This function does most of the heavy lifting. Activating and deactivating wallrunning based on a bunch of prerequisites. These can be found in the “Check Can Wallrun” Macro, which looks like this:

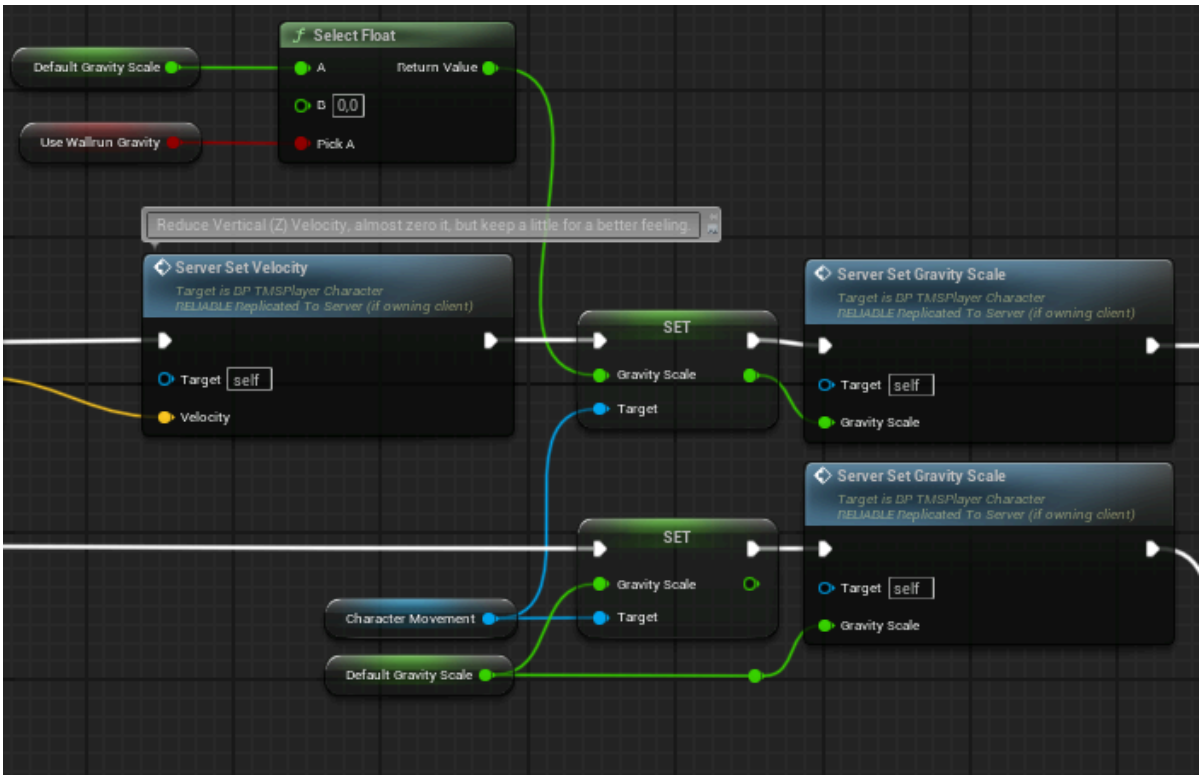


It checks for things like: having a wall on either side, not being too close to the ground, being in the air, not climbing, holding the W key and utilizing the “Check Wall Blacklist” Macro, which looks like this (Macro in Macro :p):

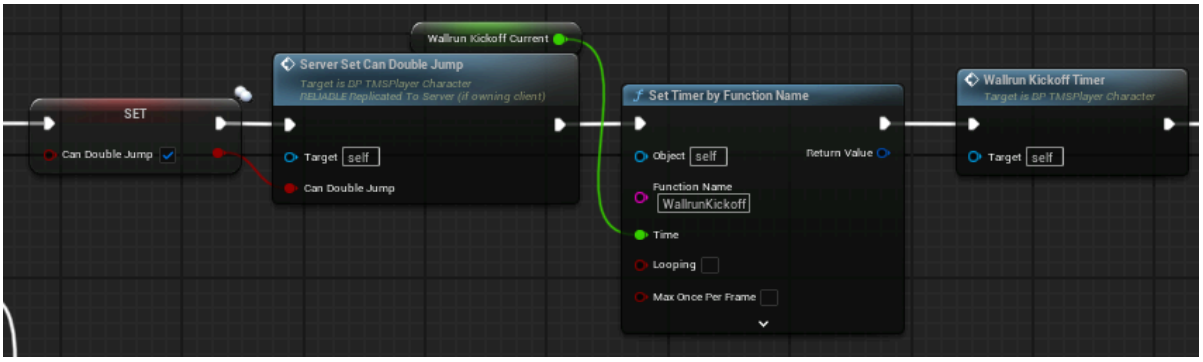


This Macro might not make a whole lot of sense yet, but in a nutshell, it checks for the last wall we have touched. In Titanfall 2, you CANNOT grab the same wall twice, UNLESS you grab it at a lower point. This makes scaling up the same wall pretty much impossible and incentivises using the climbing system instead. The “Wallrun Blacklist” Structure is set whenever we stop wallrunning, and stores the reference of the last wall we were on, the height we stopped wallrunning and if the wall is regrabbable in the first place.

Moving along in the Check Wallrunning function, we arrive at the following point:

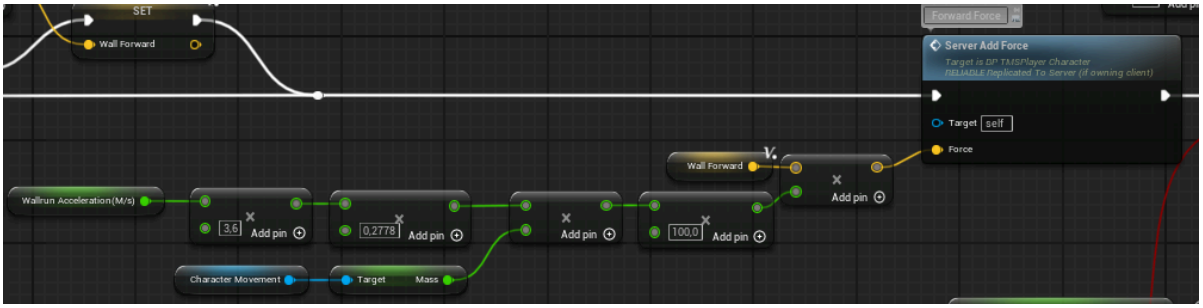


This point decreases (or resets if we stopped wallrunning) our gravity scale, unless specified that we want to use gravity while on a wall. In Titanfall 2, the character experiences no gravity while wallrunning, which is why this is disabled by default.

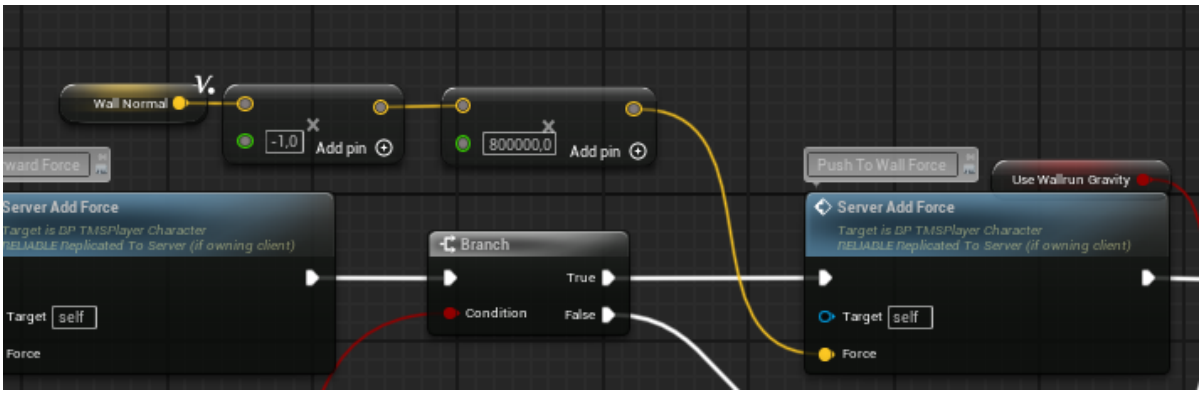


We then reenable double jumping, as if we landed, and start the Kickoff Timer, which forces the player off the wall he's on if he runs on it for longer than the specified time.

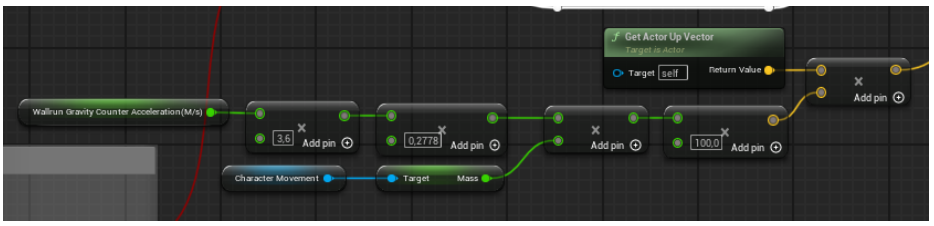
Wallrunning Movement:



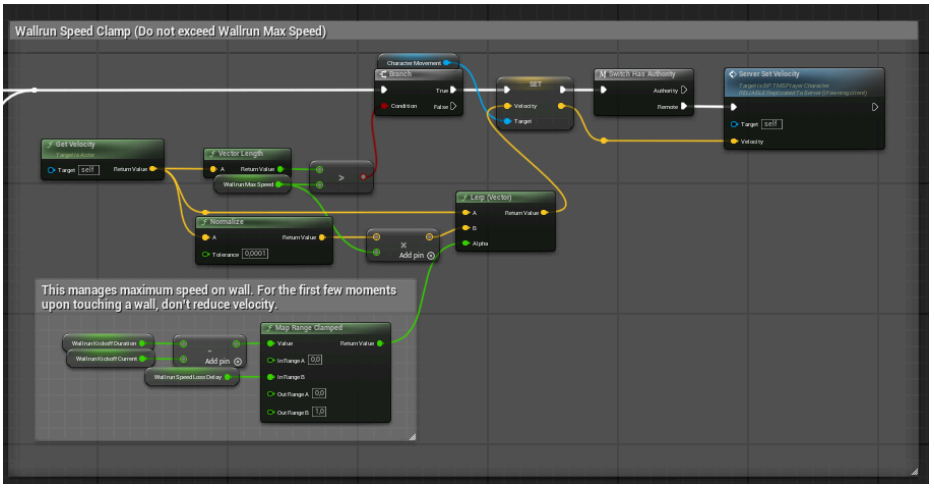
While this isn't the start of the wallrunning movement function, it is the first calculation we do for the movement along the wall. We scale up the "Wallrun Acceleration (m/s)" variable to centinewtons(cN), which is the unit of force in unreal engine. This moves us along the wall, accelerating us to the "Wallrun Max Speed" speed, but not past it.



We also push the player towards the wall.



If we are using wallrun gravity, this calculation here adds a counterforce to our player, so that gravity doesn't pull us down as much.



This final portion is what is responsible for limiting the players velocity. We also don't want to immediately limit it, as if we are moving at a faster speed, so-called wallkicks can help us gain even more speed. This calculation can be seen in the bottom left.

We also tilt the camera based on which side the wall we are running alongside is on. This is handled via the "Wallrun Camera Effects" function.

This sums up the basics of the wallrunning system. The variables used to control the feel of wallrunning are the following:

Variable Name	Variable Type	Description	Default Value
bWallRunning	boolean	Keeps a reference of our character's wallrunning state.	false
bUseWallrunGravity	boolean	Enables/Disables gravity while wallrunning.	false

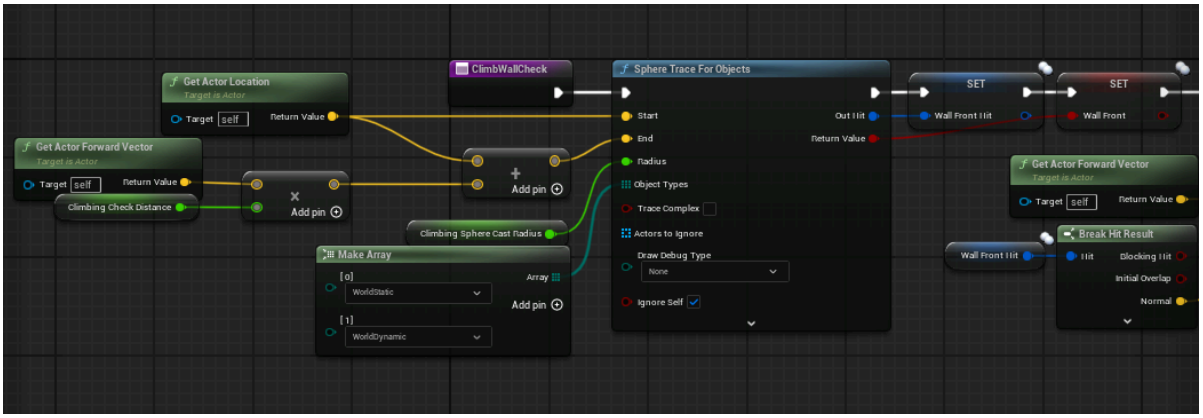
Variable Name	Variable Type	Description	Default Value
WallrunGravityCounter Acceleration(m/s)	float	The acceleration that is applied when bUseWallrunGravity is set to true. Helps reduce the gravity while on the wall.	10 m/s
WallrunKickoffDuration	float	Controls how long the player can stay on the same wall before being automatically kicked off.	1,75 secs
WallrunKickoffCurrent	float	Don’t modify. Keeps track of how long we've stayed on a wall. Only gets reset if we exit a wall and aren't wallrunning for the next 0,2 seconds.	-
WallrunMaxSpeed	float	The maximum speed you can achieve while wallrunning. Upon starting a wallrun, gradually accelerate to this speed.	1600 Units
WallrunAcceleration(m/s)	float	The acceleration the player experiences while they're wallrunning.	20 m/s
WallrunBlacklist	Structure	A struct that keeps a reference of the latest wall the player was wallrunning on. This checks for regrabbability of walls.	-
bWallLeft/bWallRight	boolean	Keeps a reference of any wallrunnable walls on our left/right.	false
WallrunCheckDistance	float	Controls how far to trace for walls to the left/right of our player.	75 Units
WallrunMinGroundDistance	float	The player needs to be at least this far above the ground to even be able to wallrun.	125 Units
WallLeftHit/WallRightHit/ CoyoteWallHit	Hit Results	Don’t modify. Hit results of any wallrunnable walls to our left/right, and the last wall hit result we had (used for coyote time).	-
WallJumpUpForce	float	The upwards launch force the player experiences when they jump off of a wall.	460 Units
WallJumpSideForce	float	The sideways launch force the player experiences when they jump off of a wall.	800 Units
bExitingWall	boolean	Keeps a reference of if our character has just stopped wallrunning. Resets after ExitWallTime.	false
ExitWallTime	float	The time the "Exiting Wall" Boolean will remain true.	0,2 secs
WallrunCameraTilt	float	Controls how much the camera tilts when wallrunning.	7,5 deg
WallrunCameraTiltInterp Speed	float	Controls how fast the camera interpolates to the tilt rotation.	10 Units
bWasWallrunning	boolean	Keeps a reference of if our character has just stopped wallrunning. Resets after a fixed 0,2 seconds.	false
WallrunSpeedLossDelay	float	This is the time it takes for a wallrun to slow the player down to "Wallrun Max Speed" when they're faster. This slowdown occurs gradually. This is used to reward players using "Wallkick-Tech", which results in them keeping and in some cases gain momentum when jumping off of a wall almost immediately.	0,1 secs

Climbing and Ledgegrabbing

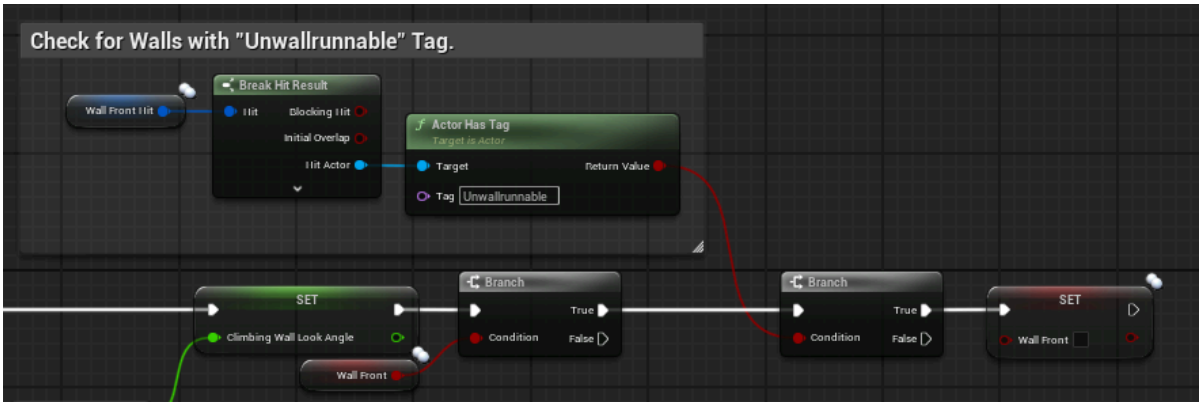
While the implementation of both systems deviates quite a little, I found it best to sum both up under one chapter. I intend to rework both systems in the future, as I am not 100% happy with the current implementation. It is quite challenging to include both climbing and wallrunning as they can very easily transition into each other unwantedly. As with the wallrunning, both climbing and ledgegrabbing functionality is handled on tick:

Let's break down the individual functions. Starting with:

Climb Wall Check:

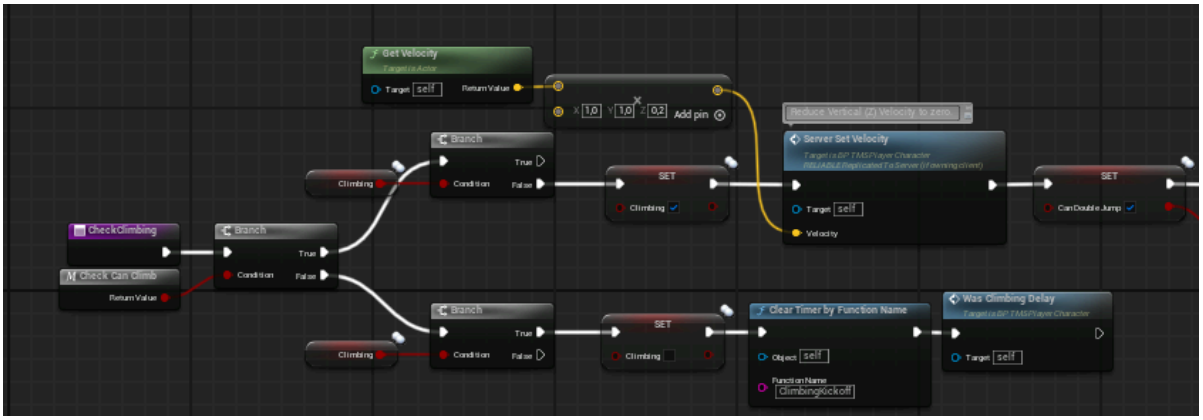


This function is very similar to it's Wallrunning counterpart "Check For Wall". However, it traces forwards instead of to the sides and uses a sphere trace instead of a line trace. This prevents the climbing from stopping immediately when the player looks over a ledge.



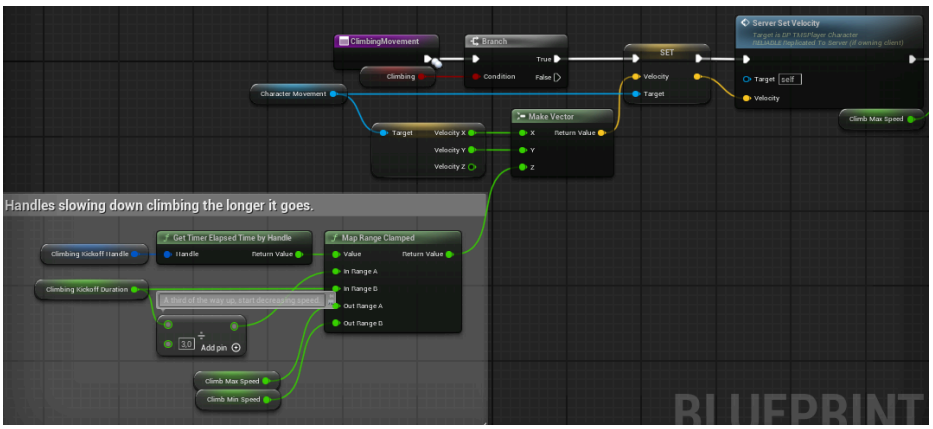
As we did previously, we check to see if the wall in front of us is tagged as “Unwallrunnable”, excluding it from the climbable walls. We also calculate the angle between our look direction and the wall. Climbing is only possible if we look directly at the wall, with a bit of leeway.

Check Climbing:



This function is essentially a one-to-one copy of the “Check Wallrunning” function. It checks a list of prerequisites and enables/disables climbing based on it. It also resets double jumping and starts the Climbing Kickoff Timer.

Climbing Movement:



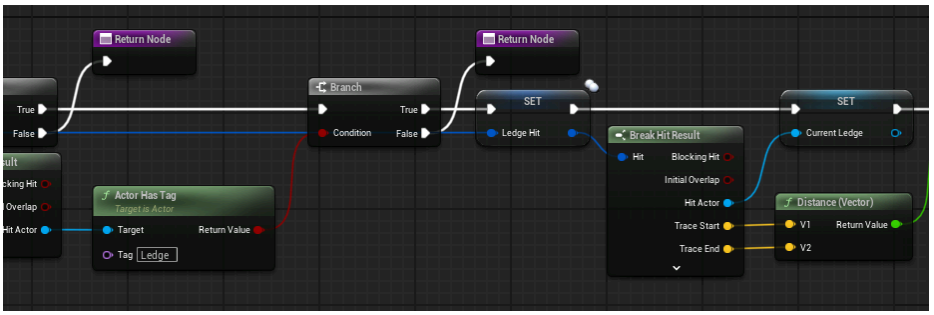
The climbing movement is quite simple this time around, essentially adding an upwards force while we are on the wall, which decreases the longer we climb.

That wraps up the Climbing Implementation. The variables used are the following:

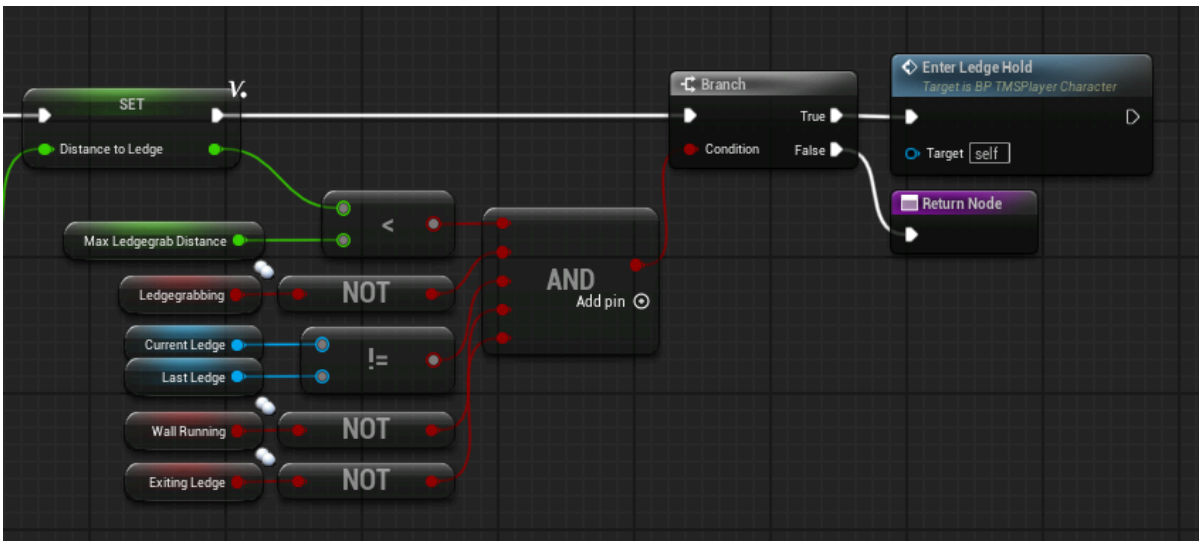
Variable Name	Variable Type	Description	Default Value
bClimbing	boolean	Keeps a reference of our character's climbing state.	false
ClimbMaxSpeed	float	The maximum speed of climbing up walls. Climbing gets progressively slower the longer you're doing it.	550 Units
ClimbMinSpeed	float	The minimum speed of climbing up walls.	350 Units
ClimbingKickoffDuration	float	How long the player can climb on the same wall before being automatically kicked off.	1,5 secs
ClimbingCheckDistance	float	Controls how far to trace for walls in front of our player.	75 Units
ClimbingSphereCastRadius	float	Controls the radius of the sphere when tracing for climbable walls.	25 Units
ClimbingMaxWallLookAngle	float	The maximum angle the player can look away from the wall to still be able to climb. Shouldn't be greater than 45 degrees.	30 deg
ClimbingWallLookAngle	float	Don't modify. The current wall look angle.	-
ClimbingSidewaysMovement Multiplier	float	The movement multiplier for sideways movement while climbing up a wall. Limits diagonal movement.	0,1 (10%)
bWallFront	boolean	Keeps a reference of any climbable walls in front of the player.	false
WallFrontHit	Hit Result	Don't modify. Hit result of any climbable wall in front of the player.	-

Variable Name	Variable Type	Description	Default Value
ClimbJumpUpForce	float	The upwards launch force the player experiences when they jump off of a wall while climbing.	460 Units
ClimbJumpBackForce	float	The backwards launch force the player experiences when they jump off of a wall while climbing.	800 Units
bExitingWallClimbing	boolean	Keeps a reference of if our character has just stopped climbing. Resets after ExitWallTimeClimbing.	false
ExitWallTimeClimbing	float	The time the "Exiting Wall Climbing" Boolean will remain true.	0,5 secs
ClimbingBlacklist	Structure	A struct that keeps a reference of the latest wall the player was climbing on. This checks for regrabbability of walls.	-
ClimbingRegrabLowerHeight	float	The height subtracted from Regrab Height. Forces players to only be able to climb the same wall when either touching the ground or grabbing at a significantly lower point .	800 Units
ClimbingMinGroundDistance	float	The player needs to be at least this far above the ground to even be able to climb.	75 Units
ClimbingKickoffHandle	Timer Handle	Don't modify. Is the handle which keeps a reference of the current climbing duration. Is used for calculating climbing speed decrease the longer we climb.	-
bWasClimbing	boolean	Keeps a reference of if our character has just stopped climbing. Resets after a fixed 0,2 seconds.	false

Now that we have finished climbing, let's quickly go over the ledgegrabbing implementation.

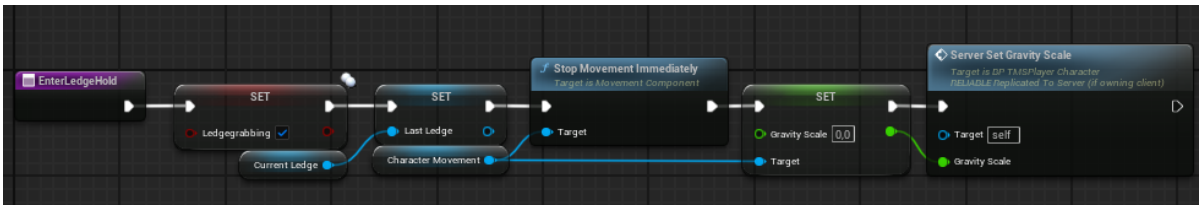


The testing for any ledges is very similar to the climbing detection, so I will not go over that in full detail. Most importantly, we check to see if the ledge we hit has the tag "Ledge". I want to revamp this system in the future to dynamically detect ledges during climbs, but for now this implementation has to do!



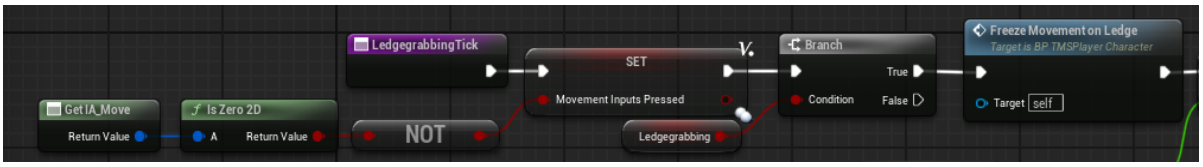
If these prerequisites are met, we “Enter Ledge Hold”.

Enter Ledge Hold:

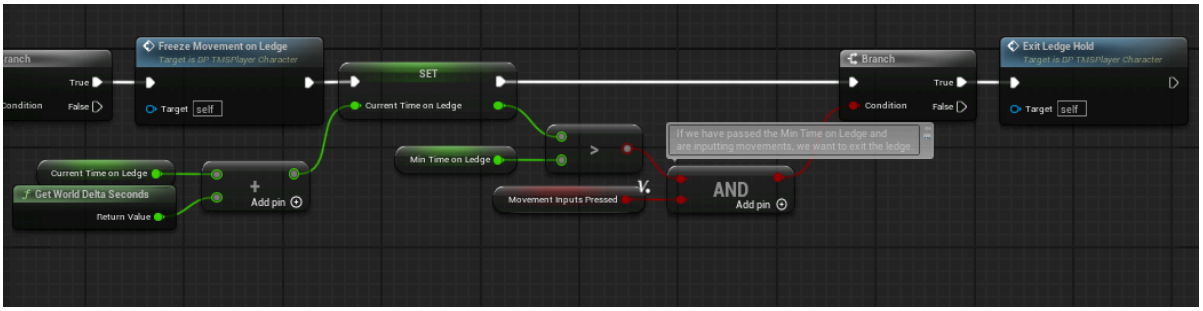


This function is very basic, mainly setting ledgegrabbing to true, stopping all movement, storing a reference of the ledge, and also disabling gravity.

Ledgegrabbing Tick:

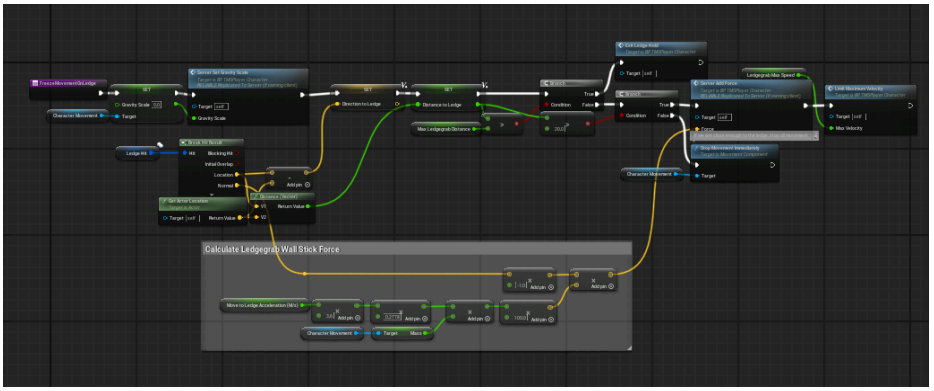


This part of the function freezes the movement of the player while they’re holding onto a ledge, and also checks if the player is pressing any movement keys.



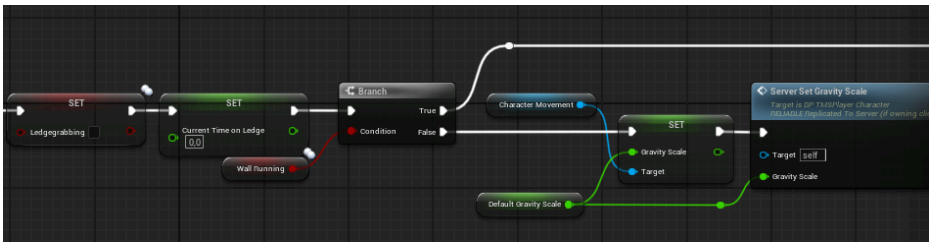
If we are pressing movement keys, and the time we held onto a ledge exceeds the “Min Time on Ledge” variable, we can let go, and we “Exit Ledge Hold”.

Freeze Movement on Ledge:



This function isn’t really complex. It mainly sticks the player to the ledge by applying a force, and also continuously sets the gravity scale to 0. It also checks if our player – for whatever reason – is too far away from the ledge, and exits ledgegrabbing if they are.

Exit Ledge Hold:



This function mainly resets the “bLedgegrabbing” boolean, the “Current Time on Ledge” variable and resets gravity if we aren’t wallrunning.

That concludes the Ledgegrab Implementation. The variables used are the following:

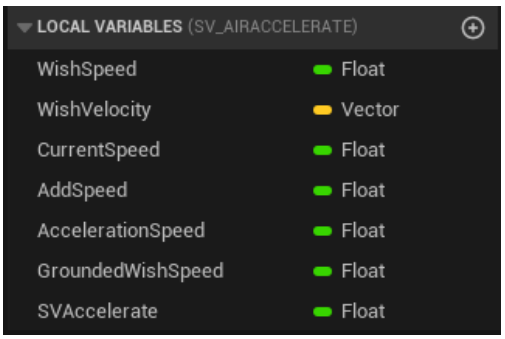
Variable Name	Variable Type	Description	Default Value
bLedgegrabbing	boolean	Keeps a reference of our character’s ledgegrabbing state.	false
LedgegrabMaxSpeed	float	The maximum speed of being pulled towards a ledge.	800 Units
MoveToLedgeAcceleration	float	The acceleration towards the ledge the player experiences	20 m/s

Variable Name	Variable Type	Description	Default Value
(m/s)		while they're ledgegrabbing.	
MaxLedgegrabDistance	float	Controls how far away the player can be from a ledge before we stop grabbing entirely.	75 Units
LedgegrabCheckDistance	float	Controls how far to trace for ledges in front of our player.	75 Units
LedgegrabSphereCast Radius	float	Controls the radius of the sphere when tracing for ledges.	25 Units
LedgeHit	Hit Result	Don't modify. Hit result of any ledge in front of the player.	-
MinTimeOnLedge	float	The minimum duration the player is locked to a ledge before they can jump or move off.	0,5 secs
CurrentTimeOnLedge	float	Don't modify. Keeps track of how long we've been holding onto a ledge.	-
CurrentLedge/LastLedge	Actor	Keep references of the current and last ledge we've been holding onto.	-
LedgegrabJumpUpForce	float	The upwards launch force the player experiences when they jump off of a ledge.	600 Units
LedgegrabJumpBackForce	float	The backwards launch force the player experiences when they jump off of a ledge.	200 Units
bExitingLedge	boolean	Keeps a reference of if our character has just stopped ledgegrabbing. Resets after ExitLedgeTime.	false
ExitLedgeTime	float	The time the "Exiting Ledge" Boolean will remain true.	0,5 secs

As stated before, these systems are at the top of my rework list. If you have any suggestions please write them in the Discord suggestions channel or contact me directly.

Source-Like Air Acceleration

This is arguably the most mathematical mechanic in TMS. I will not go over the calculations, if you're really interested in them, check out both the "Source Air Acceleration" and "SV_AirAccelerate" functions. The "SV_AirAcceleration" function contains a lot of local variables, most of which are calculated on their own.



The exception to this is the SVAccelerate variable, which has a default value of **2!** This value controls the strength of airstrafing. In heavy contrast to other Source/Quake Engine titles where this value is 10, Titanfall 2 has this value set to 2. This makes airstrafing a lot slower than in for example *Half-Life,* while still allowing for preservation of speed.

Airstrafing allows players to move in a curve through the air. This simple mechanic single-handedly made players fall in love with source and quake engine movement, and is still one of the most notorious mechanics in gaming to this day. I found it very important to include it in this system, since it gives players a lot more control and maneuverability while airborne.

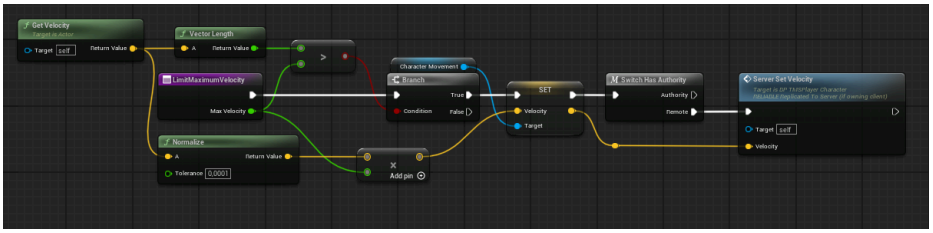
What's next?

With the air acceleration explained, that wraps up the movement mechanics of the Player Character. Before we talk about the Grappling Hook, I'd like to add a few overarching functions and variables inside of the BP_TMSPlayerCharacter. These include:

Variable Name	Variable Type	Description	Default Value
SpeedCap	float	The maximum velocity the character can have. Can be overwritten by grappling.	2500 Units
StandingCapsuleHalfHeight	float	The half height of the character's capsule when standing.	88 Units
CrouchingCapsuleHalfHeight	float	The half height of the character's capsule when crouching or sliding.	55 Units
DefaultGravityScale	float	The default gravity scale. Also set this inside of the character movement component. This is only used for resetting it to it's original value after certain actions.	1,17

Variable Name	Variable Type	Description	Default Value
DefaultJumpZVelocity	float	The default Jump Z Velocity. Also set this inside of the character movement component. This is only used for resetting it to it's original value after sliding.	575 Units
SlidehopJumpZVelocity	float	The Jump Z Velocity while sliding. We want to decrease the height of jumps during slides by just a little, to make them feel more grounded.	430 Units

Limit Maximum Velocity:

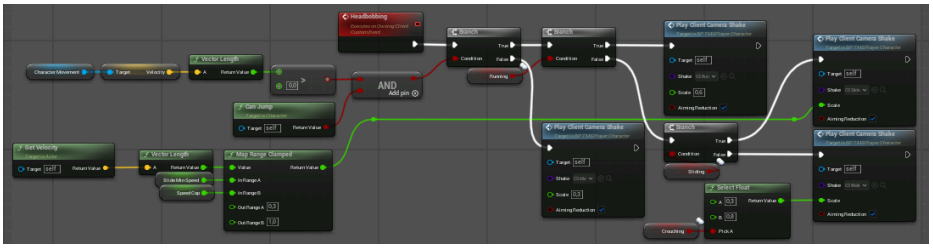


This function is used a lot, and clamps the velocity of the player to a specific value.

OwningClient_PlayCameraShake:

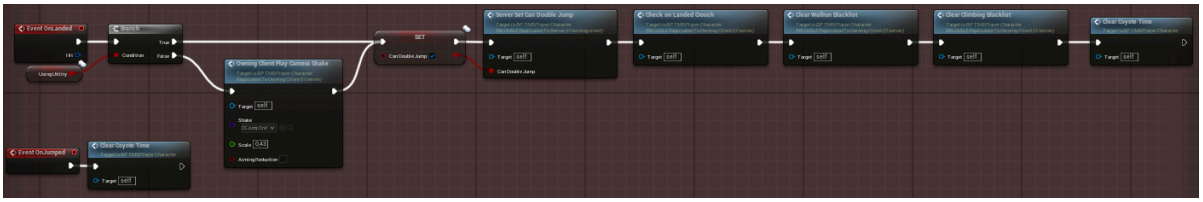
This event is helpful for playing camera shakes only on a specific client during networked play.

Headbobbing:



This event is used to simulate a walking/running/sliding camera shake. Is called on Tick and can be nauseating to some.

OnLanded Event:

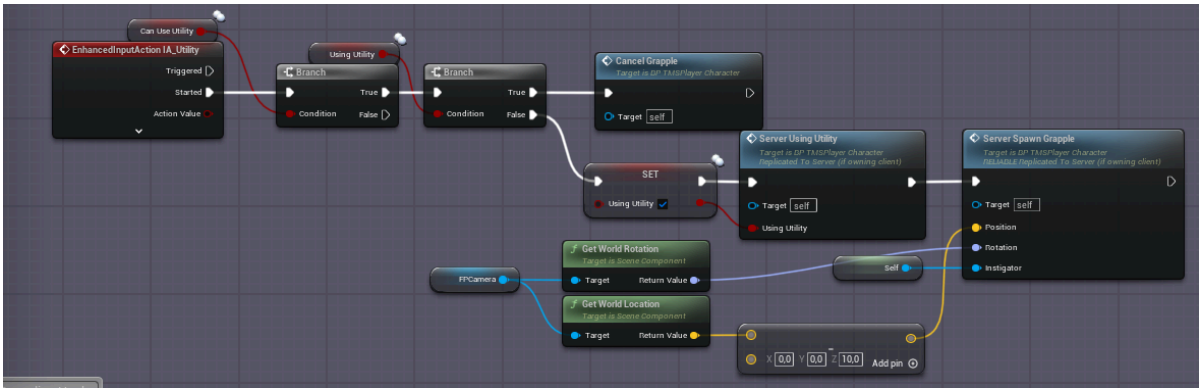


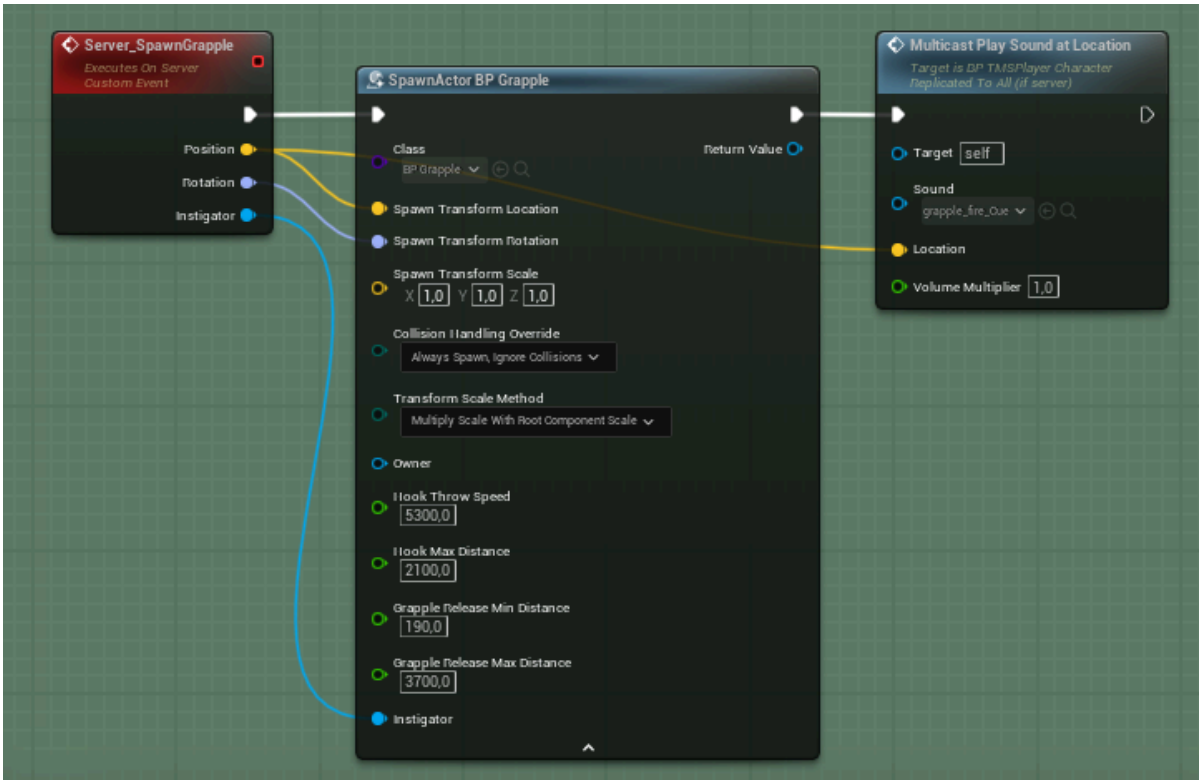
This event clears coyote time and wallrun/climbing blacklists, reenables double jumping and plays a camera shake. OnJumped in the bottom left also clears coyote time.

The Grappling Hook Actor

Another important mechanic in TMS is the grappling hook. Heavily inspired by the grapple in Titanfall 2 and Pathfinder's grappling hook in Apex Legends. Instead of just pulling you towards it, or just swinging and not pulling at all, this hook does both at the same time. It allows the player to look in a different direction, which makes them swing in that direction while still being pulled closer to the grapple point. The hook allows players to build a crazy amount of speed and momentum, which can be carried over into slides or wallkicks. I've tried to replicate the feel of the grappling hook in both games as close as possible, and I feel like it turned out pretty good.

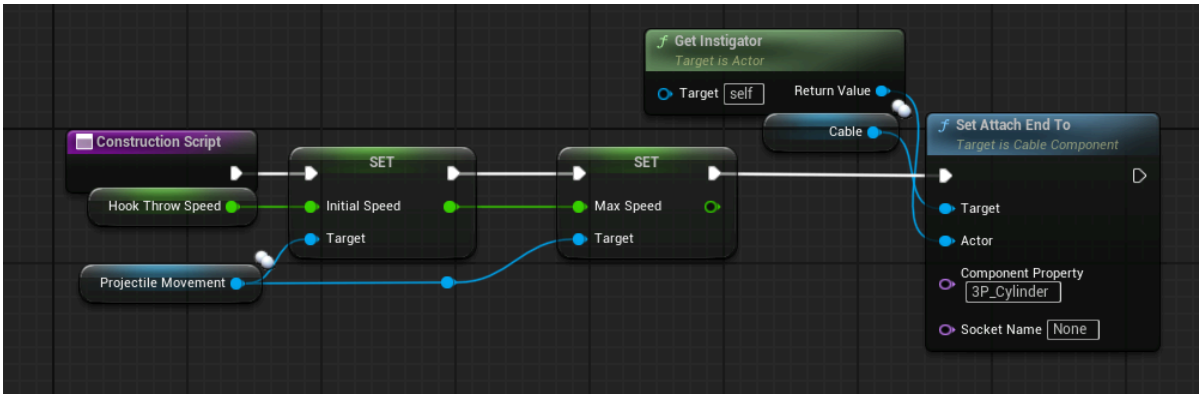
In the BP_TMSPlayerCharacter:

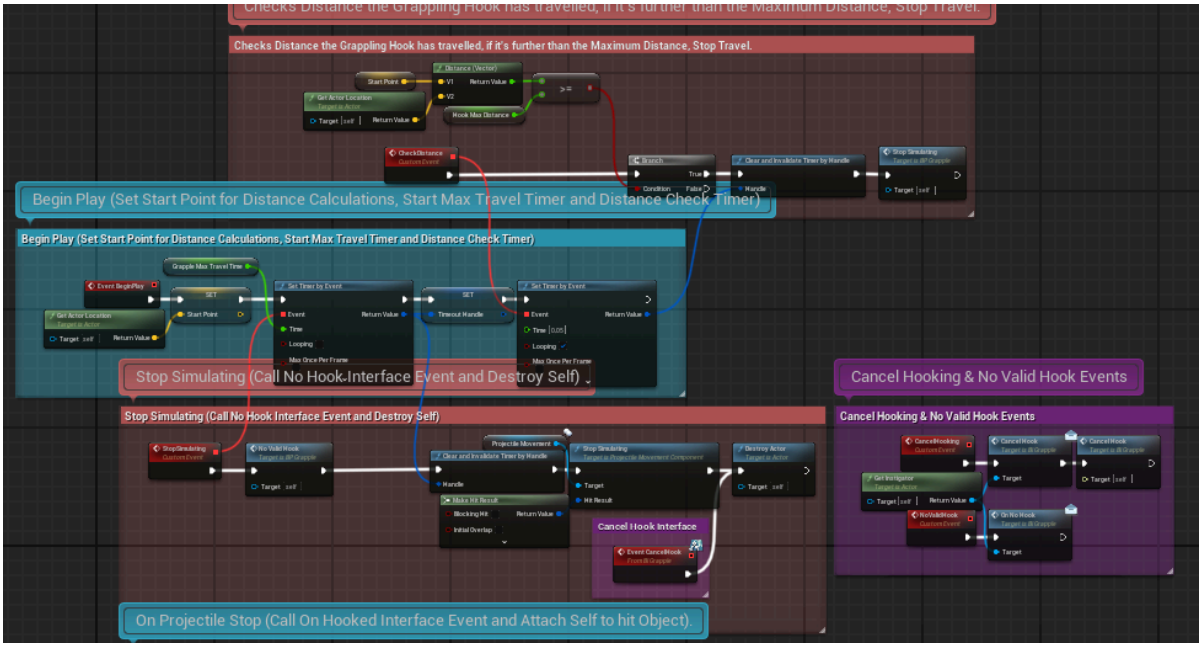




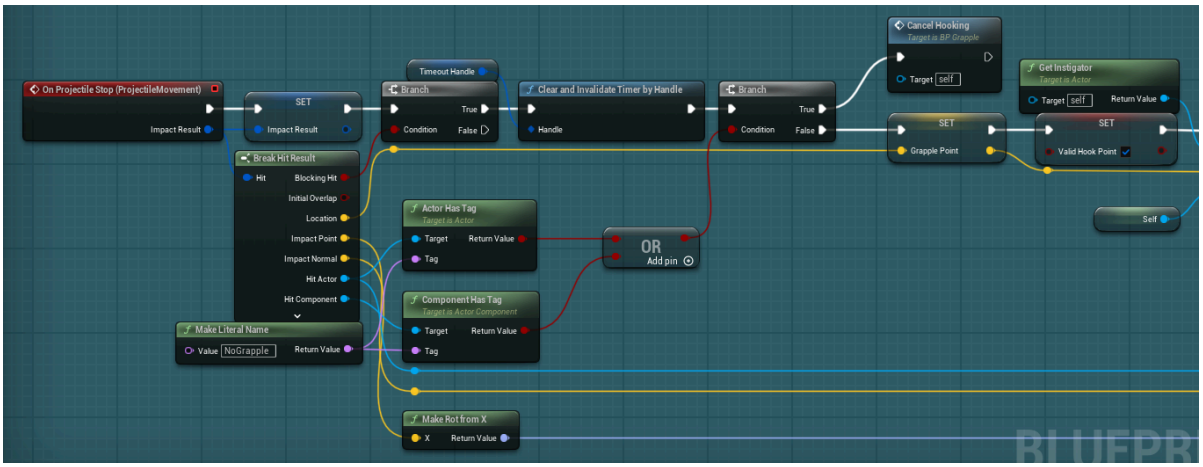
The grappling hook starts inside of the TMSPlayerCharacter blueprint, where we spawn the grappling hook on the server, and set the “Using Utility” boolean to true. If we are currently grappling and press the button again, we will cancel our grapple. We will also cancel whenever we press the crouch key while being pulled.

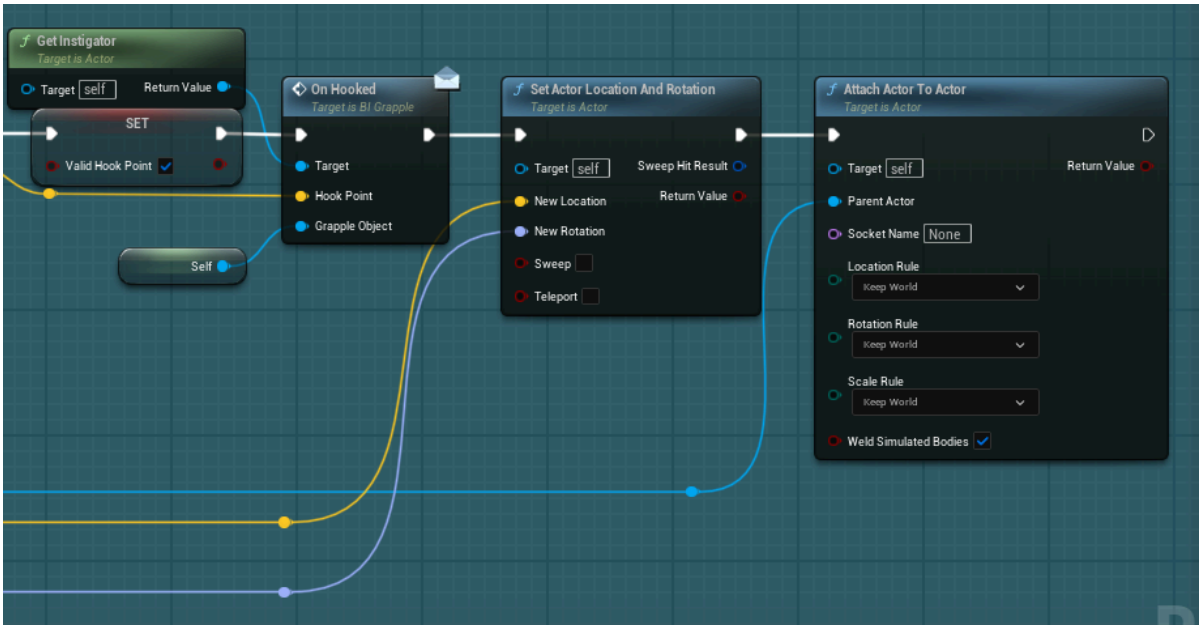
In BP_Grapple:



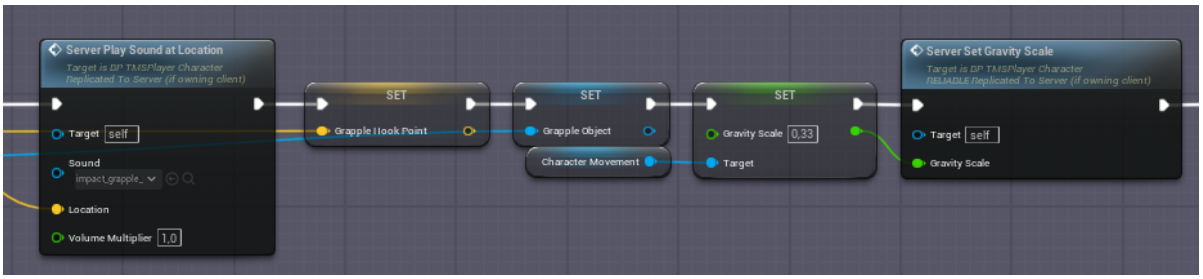


In the BP_Grapple actor, we manage the movement of the grappling hook via a ProjectileMovementComponent. We also continuously test for the distance of the hook, destroying it when it exceeds the “Hook Max Distance” variable. If we hit anything, we continue as follows:

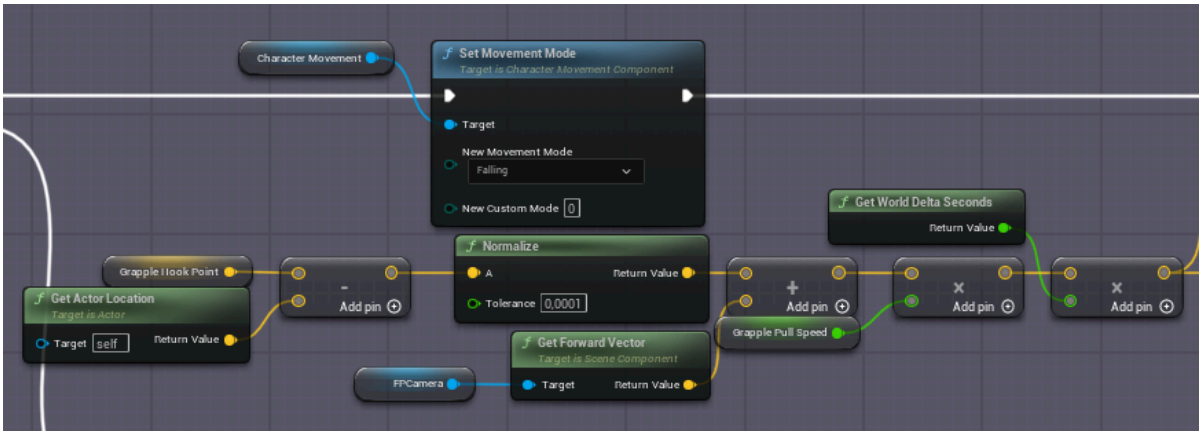




We stop any movement simulation, and call the OnHooked event via the BI_Grapple Blueprint Interface. This passed through to our player, where we do the following:



We play a replicated sound, set both the “Grapple Hook Point” and “Grapple Object” variables, decrease gravity scale and clear wallrun/climbing blacklists.



We then continuously move the player to the result of this calculation, which takes both the “Grapple Hook Point” and look direction of the player into consideration. We also test if our grappling hook is still valid, meaning we check a list of prerequisites

which include distance between us and the hook point, look angle and obstructing walls.

This concludes the grappling hook Implementation. The variables used are the following:

In BP_TMSPlayerCharacter:

Variable Name	Variable Type	Description	Default Value
bUsingUtility	boolean	Keeps a reference of our if our character is using any utility (being pulled by the grappling hook).	false
bCanUseUtility	boolean	Keeps a reference of our if our character can use the grappling hook.	true
bPulledByGrapple	boolean	Keeps a reference of if our character is actively being pulled by the grappling hook.	false
GrappleHookPoint	vector	Don't modify. Gets passed through by the BI_Grapple interface. Used for calculating pull velocity.	-
GrappleObject	BP_Grapple	Don't modify. Gets passed through by the BI_Grapple interface. Reference to our grappling hook.	-
GrapplePullSpeed	float	The speed the grappling hook pulls the player.	1000 Units
GrapplePullDuration	float	The duration that we actually get pulled along by the grapple before it automatically disconnects.	2,5 secs
GrappleUtilityCooldown	float	The cooldown between grapples.	1 sec

In BP_Grapple:

Variable Name	Variable Type	Description	Default Value
HookThrowSpeed	float	Controls how fast the hook travels. Is exposed on spawn.	5300 Units
HookMaxDistance	float	Controls how far the hook travels. Is exposed on spawn.	2100 Units
GrapplePoint	vector	Don't modify. Keeps a reference of the point where our Hook hit something. Passed through to our PlayerCharacter to pull them towards it.	-
ValidHookPoint	boolean	Used for reference if we have a valid hook point. Can be used in other Blueprints. (Unused by default).	false
StartPoint	vector	Don't modify. Keeps a reference of the location when the hook was thrown. Used for distance calculation.	-
ImpactResult	Hit Result	Keeps a reference of the Hit Result of our hook impact point. Used to check for continuous valid grapple check.	-

Variable Name	Variable Type	Description	Default Value
GrappleReleaseMinDistance	float	Cancels the grapple when the player is this close to the hook point. Is exposed on spawn.	190 Units
GrappleReleaseMaxDistance	float	Cancels the grapple when the player is this far away from the hook point. Is exposed on spawn.	3200 Units
GrappleMaxTravelTime	float	Cancels the grapple after this time has passed. Is exposed on spawn.	1,75 secs
TimeoutHandle	Timer Handle	Don't modify. Is the handle which keeps a reference of the current travel duration.	-

Conclusion

Looks like you've made it to the end of this documentation. I've really tried my best at explaining most of the mechanics and functionality that come with TMS. If you have any further questions, feel free to join the support discord server (Join Here!).

Thanks a lot for making it all the way through! Whether or not you've bought TMS or are looking through the documentation to see if it's suited for you, I hope you have a wonderful day!

~ Yannick